

F20FO/F21FO - Digital Forensics

Lab 06: Anti-Forensics & Cryptography

Lab 6: Objectives

This lab intends to introduce you to concepts of anti-forensics and cryptography. At the end of the lab you will be able:

1. Symmetric Key Encryption
2. Asymmetric Key Encryption
3. Password Cracking

Notes

There is nothing to submit at the end of this lab, however students are encouraged to log their results in a document. There are some questions **coloured in blue** that you can use as indication to record in your logbook (e.g. lab06_logbook). Often the lab instructions are intentionally open ended, and you will have to figure some things out for yourselves.

The lab will require you to install software and download additional resources from the Resources module for lab06 in Canvas. Changes to your machine should not impact on its functionality – however you are recommended to use a VM.

This lab will cover several important fundamental concepts for digital forensics and will also look at introducing you to developing your Python coding abilities to help understand these fundamentals.

Teaching you how to create your own tools in Python is a job for more than one lab. You may have already studied programming in previous courses. Use the knowledge you have gained and your general intelligence to support you in this lab. We are not going to teach you Python from scratch but will help you to develop some simple applications that will allow you to automate and verify some of the techniques you are practising.

If you do not have *Python* already installed on your machine, please download and install version 3 of Python:

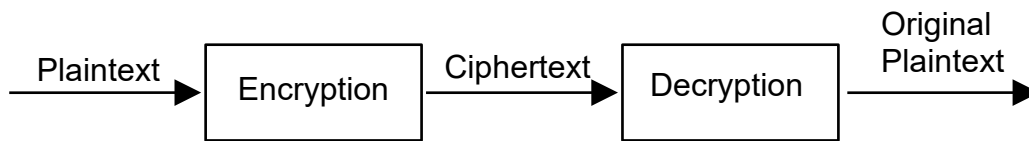
<https://www.python.org/downloads/>

After installation of Python, install *Pillow*, which is a Python library:

<https://pillow.readthedocs.io/en/stable/installation.html>

The secure communication and storage of data on a computer is a key issue for protecting sensitive data that is stored or sent through electronic communication - businesses and criminals wish to keep their data hidden and unreadable, preventing unauthorised users from viewing it.

Essentially all cryptographic systems follow this process:



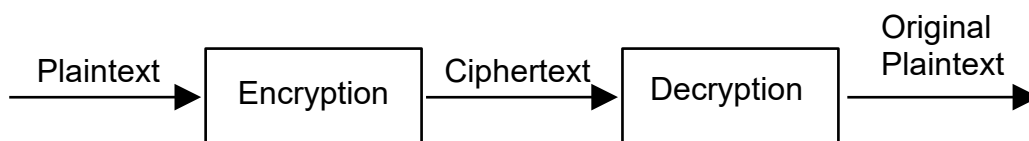
The original message is known as **plaintext** and the encrypted message is known as **ciphertext**. **Encryption** is the mathematical process of changing **plaintext** into **ciphertext**, **decryption** is the opposite. There are two main areas of encryption, which we will cover in detail in this lab:

- Symmetric Key Encryption: using the same key (password) to encrypt and decrypt data
- Asymmetric Key Encryption: using different keys to encrypt and decrypt data

Lab 6.1: Symmetric Key Encryption

This type of encryption requires that the same key is used for both encrypting and decrypting.

Key



One of the biggest security concerns with this encryption is the distribution of the key between all parties. Once someone has the key, which must be shared for others to use the encryption, then the encryption is broken.

Shift Cipher

One of the simplest forms of a symmetric encryption system is the Shift Cipher. The original, and most widely known, is the Caesar Cipher. As the name suggests this was created by Julius Caesar and was used to communicate with his generals. There was then no concern that anybody who intercepted the message would be able to decipher it.

The Shift Cipher is a simple substitution cipher that substitutes letters for other letters in the alphabet based on a given shift (or key); in the case of Caesar this was a shift of three, therefore the plaintext letter 'A' became the ciphertext letter 'D'. To decipher the message the reverse is done.

Question: Decrypt the following substitution Cipher, identify the key used, and show both the key and plaintext.

Kbtp fp fkclojxqflk xylrq zroobkq bsbkqp.

Plaintext:

Key:

Question: What does “CAESAR” become with a shift of 6?

Question: What key do we need to make “CAESAR” become “MKOCKB”?

Key:

The Substitution Cipher, swapping one letter for another, is easy to use, both for encryption and decryption, it is for this reason that the cipher is so weak. Simple guesswork can identify the encryption key.

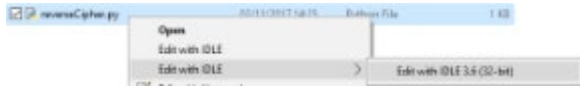
Plain	C	D	E	H	I	N	P	R	S	T	Y
Cipher	X	J	L	A	Z	E	V	K	H	O	M

Question: Given the ciphertext **JLXKMVO OAZH**, using the above table identify the plaintext.

Python - Reverse Printing

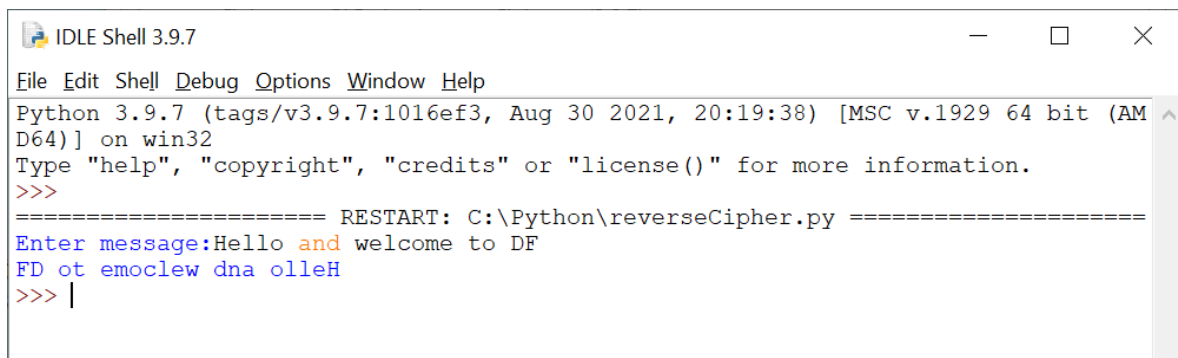
Download the code [reverseCipher.py](#) from lab06 resources folder.

Right click on the Python script file, and “Edit with IDLE” or any other preferred IDE.



To run the code, from the ‘Run’ menu select “Run module”; or press F5.

You should see the following screen:



```
IDLE Shell 3.9.7
File Edit Shell Debug Options Window Help
Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Python\reverseCipher.py =====
Enter message:Hello and welcome to DF
FD ot emoclew dna olleH
>>> |
```

Can you modify the script so that it prints out each character one at a time, in the original order, rather than reversing the input?

Question: Upload a screen shot of your code and output.

Question: This is a really simple code that simply prints a string backwards - does this offer any security?

Python - Substitution Cipher

Download the code [caesarCipher.py](#) from lab06_resources folder.

This performs the substitution cipher for you, taking the plaintext (or ciphertext) and the shift key.

It is currently set to be the Caesar Cipher, encrypt only with a shift of 3. Can you modify the code so that it asks the user:

1. Whether they want to encrypt or decrypt the message?
2. Which key they wish to use?

Question: Upload a screen shot of your code and output.

Frequency Analysis

As with all ciphers weaknesses do exist, the weaknesses of this substitution cipher is that the frequencies of letters still exist. The letters used in the English alphabet are not used equally, for example the letter 'e' is used significantly more times than 'z'. Because of this it is possible to systematically work through a ciphertext to identify the plaintext, based on frequency analysis and intuition.

Question: Review the following ciphertext and use frequency analysis to crack it – this is a trial and error process and will take you some time, depending how good you are with puzzles!

VEP HYXHLVHTP MO AWFJYFLT H RFNEPS HJNEHAPV FL VEFU ZHC FU
VEHV FV FU PHUC VM KPKMSFUP VEP IPCZMSY MS IPCNESHUP, HLY
EPLRP VEP RFNEPS HJNEHAPV. VEFU FU FKNMSVHLV, APRHWUP FO
VEP UPLYPS EHU VM IPPN VEP RFNEPS HJNEHAPV ML H NFPRP MO
NHNPS, VEP PLPKC RHL RHNWVSP VEP NHNPS, YFURMXPS VEP IPC, HLY
SPHY HLC RMKKWLFRRHVFMLU VEHV EHXP APPL PLRSCNVPY ZFVE FV.
EMZPXPS FO VEP IPC RHL AP RMKKFVVPY VM KPKMSC FV FU JPUU
JFIPJC VM OHJJ FLVM PLPKC EHLIU.

You need to start by counting the frequency of the letters and enter them into this table.

a	b	c	d	e	f	g	h	i	j	k	l	m
n	o	p	q	r	s	t	u	v	w	x	y	z

Then compare these values against the second table (which is the expected frequency of letters within English), the most frequent letters in each should be a straight swap - the more text you must look at the more accurate these straight swaps become. The remaining swaps are educated guesses.

PTO

Expected Frequency

e	t	a	o	i	n	s	h	r	d	l	u	c
12.7	9.1	8.2	7.5	7.0	6.7	6.3	6.1	6.0	4.3	4.0	2.8	2.8
m	w	f	y	g	p	b	v	k	x	j	q	z
2.4	2.4	2.2	2.0	2.0	1.9	1.5	1.0	0.8	0.2	0.2	0.1	0.1

[TIP: Copy the original encrypted text into notepad and find and replace your characters (unencrypted characters should be lowercase and encrypted uppercase - make sure you select **match case** so you don't overwrite anything by mistake)]

Vigenère Cipher

The Vigenère cipher, was invented by a Frenchman, Blaise de Vigenère in the 16th century. It is known as a polyalphabetic cipher because it uses two or more inputs to encrypt the data; in particular it uses a passphrase as well as a shift cipher.

The Vigenère cipher uses this table in conjunction with a key to encipher a message.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

If we were to encode a message using the key COUNTON, we write it as many times as necessary above our message. To find the encryption, we take the letter from the intersection of the Key letter **row**, and the Plaintext letter **column**.

Key	C	O	U	N	T	O	N	C	O	U	N	T	O	N
Plaintext	V	I	G	E	N	E	R	E	C	I	P	H	E	R
Encryption	X	W	A	R	G	S	E	G	Q	C	C	A	S	E

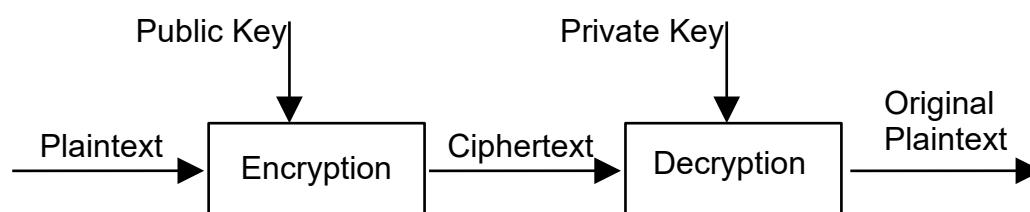
To decipher the message, the recipient needs to write out the key above the ciphertext and reverse the process.

Question: See if you can decrypt the following message that has been encrypted using the Vigenere cipher: **Mjxa xl Xxotgggm Rbrwmg**. The passphrase (keyword) used was **TCPIP**.

What are the security advantages of the Vigenère Cipher over a simple substitution cipher?

Lab 5.2: Asymmetric Encryption

This type of encryption uses a different key for encryption and decryption; it is often referred to as Public Key Cryptography. Data is encrypted with the public key and decrypted using the private key.



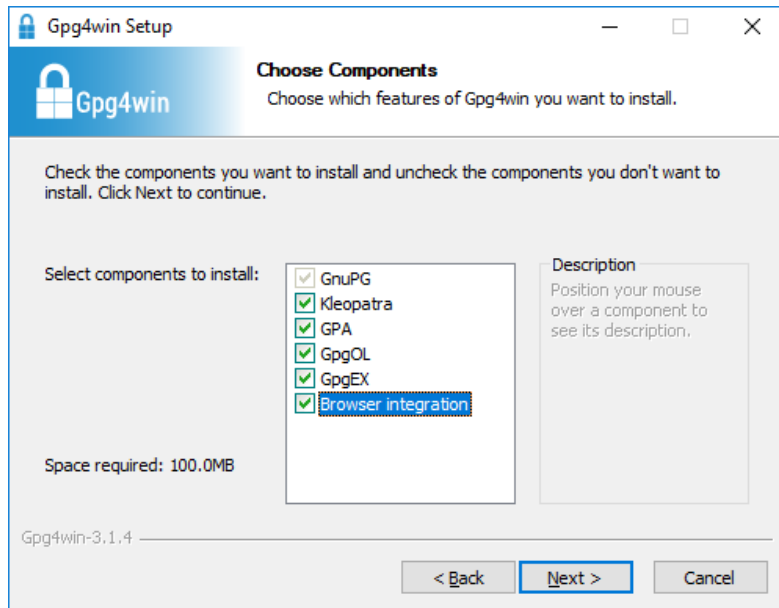
Because two keys are used, referred to as public and private keys, the issue of distributing keys is reduced. Essentially anyone in the world can have your public key and will be able to encrypt data that only the holder of the private key can decrypt.

GNU Privacy Guard (GPG)

Go to <https://www.gpg4win.org/> and read up about GPG, what it does and how it functions and then complete the following exercise.

You can access the GPG4win from the lab06_resources or download and install GPG4Win from <https://www.gpg4win.org/download.html>. (choose \$0 donation, if you want)

Install all components.



Skip configuration of certificates.

Launch GNU Privacy Assistant (GPA) .

You will be asked to generate a key now. Enter your name and email address (use your Heriot-Watt address).

Enter a passphrase – this needs to be at least 8 characters long and should contain, ideally, letters, numbers and special characters (such as “ or !).

REMEMBER YOUR PASSPHRASE AS YOU WILL NEED THIS TO USE THE APPLICATION.

You have now generated a key pair.

To distribute your public key it can be put in a public distribution location or it can be simply emailed to a recipient.

Launch Kleopatra, a sister program of GPG, which can maintain all of your public and private keys.

- Select your name in the “My Certificates” list and select Export Certificates. Save this to a folder.
- Open an email message and attach your exported certificate. Send this to someone else in the class or to yourself using your own email address (not Heriot-Watt).
- To receive a public key save the attachment to a folder. In Kleopatra select “Import Certificates” and select your saved file.
- This key will now be added to your list.
- Right click on the key in the list of imported keys in Kleopatra and select “Certify Certificate...”
- Tick the checkboxes.
- Click Next
- In the next window (step 2), select “Certify only for myself” and click Certify.
- Click Finish

You are now going to send this person a signed email and they should be able to verify that the email is from you, if they have completed the steps above to import and certify your public key.

- Compose a message to the individual you received the public key from.
- Open GPA and select Clipboard. Paste your email into this.
- Select Sign and select your name. Paste the complete contents into your email and send to the person you have shared keys with.
- Once received copy the contents of the email into GPA clipboard and select verify.

Question: What happens? What have you just achieved?

(Optional) Add other users and send emails.

Encrypt an entire message by clicking “Encrypt” in the clipboard and selecting a classmate’s public key.

Email the contents of the clipboard to your colleague and they should be able to decrypt the message using their own private key.

Question: In what circumstances might you sign using a digital signature?

Question: In what circumstances might you encrypt an entire email?

As you can see from the exercise, GPG uses a public and private key for a user. Information is encrypted with a user’s public key, which everybody can know, but is only decryptable with the user’s private key.

Some assumptions have to be made, such as only the user has access to their private key – the security of which is enforced using the passphrase.

GPG and other such tools allow secure communication, encryption of data and authentication of the message sender. All of these are intrinsic to a Forensic investigator or within an organisation that needs to keep customer data secure.

Null Cipher

A null cipher is a very basic, and early, example of steganography in which data is hidden in plain sight. Essentially the hidden message is placed inside a 'normal' message - so long as you know how to read it you can interpret the information - otherwise it looks like an innocent message.

If I was a spy awaiting news about an 'operation' I monitor websites and forums for a message that gave me the 'go'.

News Eight Weather: Tonight increasing snow. Unexpected precipitation smothers eastern towns. Be extremely cautious and use snowtires especially heading east. The [highway is not] knowingly slippery. Highway evacuation is suspected. Police report emergency situations in downtown ending near Tuesday.

Taking the first letter in each word successively yields the real message:

"Newt is upset because he thinks he is President."

You can also choose to instead use the last letter of every word, or a pattern such as 1, 2, 3 (each letter in each word).

Susan sAys Gall Lies. MAtt leTs Susan fEel joVial. Elated/aNgry?

"Sail at seven."

As you can see that so long as you have the key it is possible to read the messages, try the following.

Question: Can you work out the null cipher message? It uses a fixed point in each word.

The hairy elephant, came and took salt and tea over near the hazardous energy making apparatus today

Word Shifting

Even the layout of a document can be used to hide a message within a sentence/paragraph. Take the following two messages:

Message 1

We explore new steganographic and cryptographic algorithms and techniques
throughout the world to produce wide variety and security in the electronic web
called the Internet.

Message 2

We explore new steganographic and cryptographic algorithms and techniques
throughout the world to produce wide variety and security in the electronic web
called the Internet.

Is there any difference between the two of them?

Look closely, you will see an extra space before the words *explore*, *the*, *world*, *wide*, and *web*. Close inspection reveals the hidden message is: *explore the world wide web*. (Hint: you might need to copy and paste the message in a text editor to find the extra space)

Question: Can you work out the word shifted message below?

One reason people lie is to achieve personal power. Achieving personal power is helpful for someone who pretends to be more confident than he really is. For example, one of my friends threw a party at his house last month. He asked me to come to his party and bring a date. However, I didn't have a girlfriend. One of my other friends, who had a date to go to the party with, asked me about my date. I didn't want to be embarrassed, so I claimed that I had a lot of work to do. I said I could easily find a date even better than his if I wanted to. I also told him that his date was ugly. I achieved power to help me feel confident; however, I embarrassed my friend and his date. Although this lie helped me at the time, since then it has made me look down on myself.

Lab 5.3: Password Cracking

Traditionally passwords have been stored within the systems they are defending, making them extremely vulnerable to attack. Storing passwords was hence phased out and modern systems use more sophisticated methods of manipulating and matching passwords. For example, Linux do not store passwords but rather a hash that is generated from the password. In Windows the hashes are all stored in the directory **system32\config\SAM**. You can't see the contents of the file just by

merely opening it, but this is used to match the passwords you entered with the list of hashes stored here. Similarly, Linux based systems, store the hash under **etc/passwd** file or **etc/shadow**.

The hashing method used, is dependent on the OS. Some OS variants also use modified forms of well-known hashing algorithms. Windows NT based systems use two hashing schemes LMHash (also known as LANMAN) and NT hash. There are known flaws with both.

It might come as a surprise to you that Windows NT converts the full password into a hash. Ideally, a **salt** is used to pad the original input. It is an extra string that is added to the original input to increase the hash strength. Without **salt**, a password generation mechanism becomes vulnerable to brute-force attacks. Even with salt, brute force will work given enough time.

John the Ripper

We will use **John the Ripper** which is popular for Linux password cracking, although you could also use it on Windows systems.

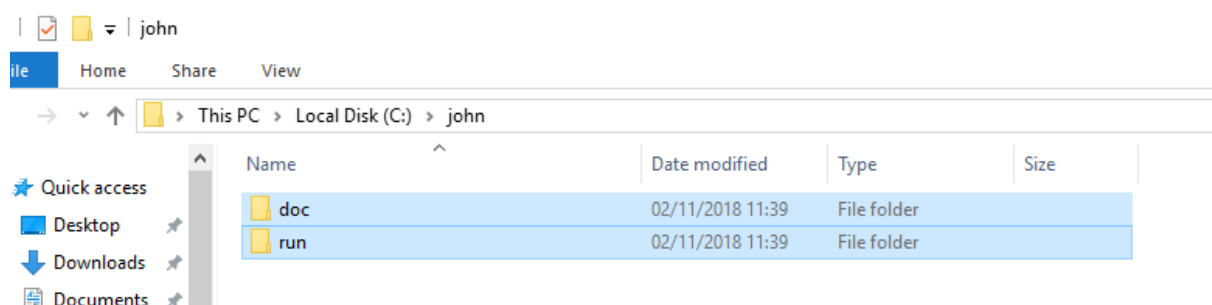
We will take a case of using John the Ripper against DES. John works very effectively in cracking DES.

Now let us generate a DES hash for some random string. There are many free tools available on the internet. Alternatively, you could use an online resource like

<https://hash.online-convert.com/des-generator>

Access John the Ripper from the lab06 resources or download **John the Ripper** from <https://www.openwall.com/john/> (Do not download the PRO version). Read the readme.md file.

Create a new folder and call it john. Unzip the downloaded file and copy the two folders doc and run into **john** folder (make sure all the files are in here).



Go back into the online DES generator link and create a new DES hash for the phrase: 1234. Copy the hash into a text file and make it look like a password file, with a colon separating the “user name” and the hash. So your file might look like

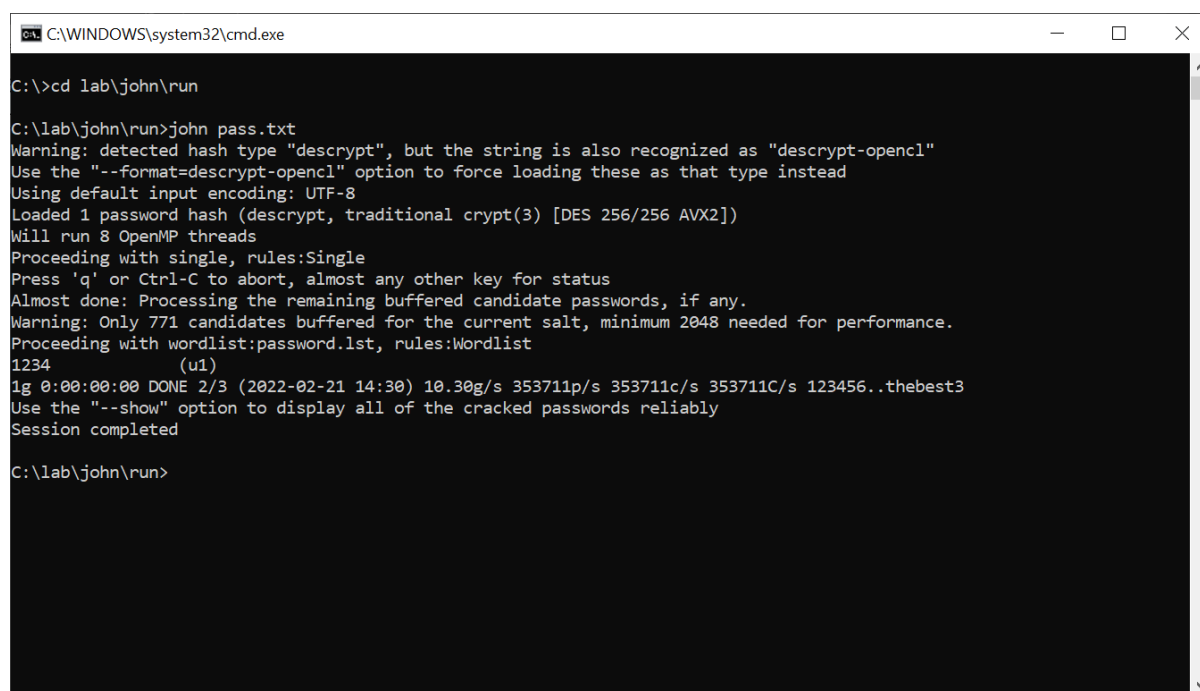
u1:CRP0W3oTu3Z4s

Save the above as a txt file under the **john\run** directory. Let us say you called the file **pass.txt**

Launch **Command Prompt (keep windows Key + R)** and type **cmd**, browse to **john\run**.

Now try out a simple brute-force method. Type in

john pass.txt



```
C:\WINDOWS\system32\cmd.exe
C:\>cd lab\john\run
C:\lab\john\run>john pass.txt
Warning: detected hash type "descrypt", but the string is also recognized as "descrypt-opencl"
Use the "--format=descrypt-opencl" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (descrypt, traditional crypt(3) [DES 256/256 AVX2])
Will run 8 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Warning: Only 771 candidates buffered for the current salt, minimum 2048 needed for performance.
Proceeding with wordlist:password.lst, rules:Wordlist
1234      (u1)
1g 0:00:00:00 DONE 2/3 (2022-02-21 14:30) 10.30g/s 353711p/s 353711c/s 353711C/s 123456..thebest3
Use the "--show" option to display all of the cracked passwords reliably
Session completed
C:\lab\john\run>
```

You should be able to see that the password cracker has found the password (see the above screenshot). In this case it might have taken only a small amount of time. However it is not uncommon for the cracker to take a long time depending on password complexity. John does not give you an update of how far it is done guessing your password. So you can just patiently wait for it to finish or narrow the guessing options if you know the nature of your password. For example, if you know that the password just contains characters, then try

john -i:alpha pass.txt

If you think it is just numbers then you can also use:

john -i:digits pass.txt

or

john -i:all pass.txt for matching against characters and numbers.

NOTE : Once a password has been guessed, John will not decode it again. If you would like to use the same hash file, then delete the **john.pot** file in the **run** directory.

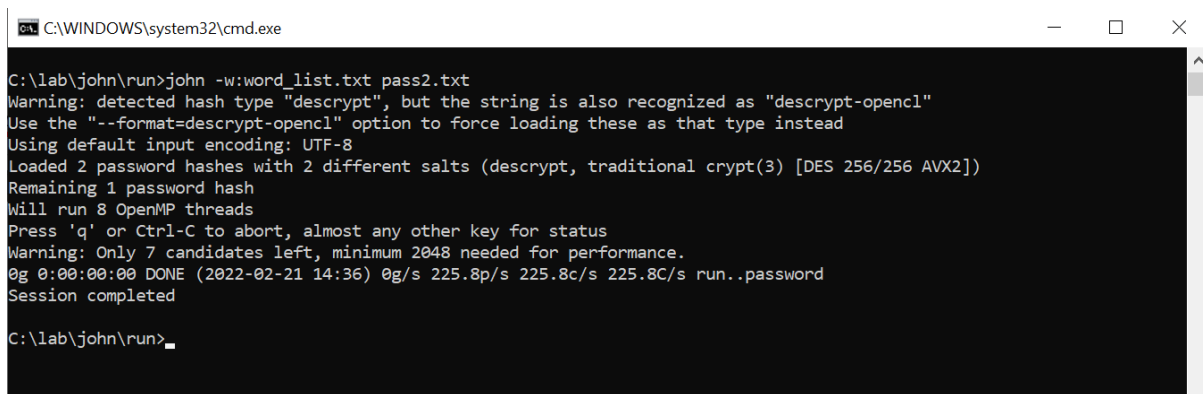
Dictionary Attack

What you have done above, is a brute-force attack. When password lists get complicated (list length and individual password length) it is not advisable to do a brute-force method. We could use dictionary attack where a word-list can be used as a parameter. Different permutations and combinations of these words will be matched against the hash. Assume that you have a wordlist (you can create your own, or there are vendors who can sell you a huge word-list!!)

For the sake of a simple example, create a hash like before. Also create another text file with a list of words you think would be most appropriate (such as the likes and dislikes of the person (or numbers they like)). Let's say your word list is called **word_list.txt** and your hash file is **pass2.txt**. Now try the following:

john -w:word_list.txt pass2.txt

The **w** option is to load a word list.



```
C:\WINDOWS\system32\cmd.exe
C:\lab\john\run>john -w:word_list.txt pass2.txt
Warning: detected hash type "descrypt", but the string is also recognized as "descrypt-opencl"
Use the "--format=descrypt-opencl" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (descrypt, traditional crypt(3) [DES 256/256 AVX2])
Remaining 1 password hash
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 7 candidates left, minimum 2048 needed for performance.
0g 0:00:00:00 DONE (2022-02-21 14:36) 0g/s 225.8p/s 225.8c/s 225.8C/s run..password
Session completed
C:\lab\john\run>_
```

You have tried out John the Ripper with DES. You can use John with different formats of hashes. For example

john --format=raw-md5 pass3.txt

Question: Generate an MD5 hash and try to crack one or more passwords. Upload screenshots of your MD5 hash and John output for the cracked(?) password(s).

End of Lab Sheet